

Memoria descriptiva — RedEconómica v1.0.0

Software educativo para la comprensión del intercambio, la especialización y la cooperación económica en niños de 8 a 12 años.

Paquete: `com.educalab.redeconomica` · Versión 1.0.0 · Android nativo (Kotlin + Jetpack Compose) · Funcionamiento 100 % sin conexión.

1. Qué es RedEconómica

RedEconómica es una aplicación Android en la que un niño entra a vivir en el **Valle Económico**, un pueblo pequeño con ocho habitantes que saben hacer cosas distintas y necesitan cosas distintas. El niño no es un espectador: es un vecino más. Produce, propone tratos, organiza el trabajo del pueblo, reparte lo que no llega para todos y decide qué hacer con recursos limitados.

La aplicación se construyó alrededor de una idea: **la economía se entiende haciéndola**. El orden es siempre el mismo:

1. **Ocorre algo.** «Tienes cinco manzanas y ningún pan.»
2. **El niño actúa.** Propone un trueque, asigna un oficio, reparte tres panes entre cinco vecinos.
3. **Hay una consecuencia medida.** El vecino acepta o no, y se explica por qué.
4. **Solo entonces aparece la palabra.** «Eso que acabas de hacer se llama intercambio.»

Nunca al revés. Ninguna pantalla empieza con una definición.

2. El mundo: el Valle Económico

El pueblo tiene trece zonas con función económica propia: Granja, Huerto, Panadería, Taller, Carpintería, Pesquería, Telar, Mercado, Plaza central, Centro de Intercambio, Cooperativa, Laboratorio y Almacén. Las zonas se abren a medida que avanza la historia.

Viven allí ocho habitantes, cada uno con **productividad real** (cuánto saca en un turno de cada producto), inventario y necesidades:

Habitante	Oficio	Produce mejor	Suele necesitar
Lía	Fruticultora	Manzanas (6/turno)	Pan, herramientas
Tomás	Panadero	Pan (5/turno)	Manzanas, madera
Nina	Carpintera	Madera y tablas	Pan, tela
Bruno	Hortelano	Verduras (6/turno)	Herramientas, pan
Sofía	Herrera	Herramientas y clavos	Verduras, madera
Emi	Pescadora	Pescado (5/turno)	Pan, tela
Dani	Tejedor	Lana y telas	Pescado, fruta
Rita	Transportista	Casi nada, pero lo mueve todo	Verduras, clavos

Esos números no son decoración: son la entrada de los motores de especialización, de ventaja comparativa y de costo de oportunidad.

Acompaña al niño **Tilo**, un zorro cartero que trae las misiones, celebra los avances y explica los errores en una frase. Aparece cuando aporta algo y calla el resto del tiempo.

3. Qué aprende y en qué orden

La progresión pedagógica tiene siete niveles, repartidos en catorce misiones:

Nivel	Idea	Misiones
1	Cada persona necesita cosas distintas	1
2	No siempre tenemos lo que necesitamos	1, 5
3	Podemos cambiar lo que nos sobra por lo que nos falta	2, 4
4	Concentrarse en una tarea produce más	3, 11
5	Coordinándonos llegamos donde solos no llegamos	7, 8, 13
6	Elegir una cosa es renunciar a otra	6, 12
7	Todo junto: una economía pequeña funcionando	9, 10, 14

Las catorce misiones, en orden: *La primera cosecha · ¿Quién tiene lo que necesito? · Cada uno a lo suyo · Intercambio de vecinos · El problema del pan · La gran decisión · Juntos podemos · La cooperativa · El mercado del Valle · La gran feria · El taller de los oficios · El invierno se acerca · Las cadenas del Valle · El Valle entero.*

4. Las ocho mecánicas

Está prohibido, por diseño, que la experiencia sea una sucesión de preguntas con cuatro opciones. De los 42 desafíos, solo 4 (el 9,5 %) consisten en valorar una propuesta; los otros 38 exigen manipular, organizar o construir:

Mecánica	Qué hace el niño	Desafíos
Intercambio	Pone productos en la mesa y propone un trato	8
Elige tu oficio	Asigna una tarea a cada habitante y ejecuta el turno	9
Trabajo en equipo	Coloca a cada uno en una etapa de la cadena	5
No alcanza para todos	Reparte un recurso escaso entre varios vecinos	6
La gran decisión	Gasta un presupuesto limitado y ve a qué renuncia	4
¿Aceptarías?	Valora una propuesta y decide	4
Lo que dejas de hacer	Mueve el reparto del turno y compara los dos totales	3
Cadena de producción	Ordena los pasos por los que pasa un producto	3

Además hay dos zonas de juego libre sin puntuación: el **Mercado del Valle** (mirar quién necesita qué y ver qué tratos serían posibles) y el **Laboratorio del Valle** (cambiar variables, ejecutar y comparar dos experimentos).

5. Los motores: por qué esto no es un guion

Toda la economía de la aplicación vive en `domain/engine`, en Kotlin puro sin una sola dependencia de Android. La interfaz **no decide nada**: pregunta.

- **TradeEngine** decide si un vecino acepta un trato aplicando siete reglas en orden: la propuesta está completa, quien propone tiene lo que entrega, quien recibe tiene lo que se le pide, lo ofrecido le sirve para alguna necesidad suya, no se queda sin algo que necesita con urgencia, no entrega mucho más valor del que recibe, y (opcionalmente) quien propone también gana algo. Cuando dice que no, dice **cuál** de las siete reglas falló y lo traduce a una frase que un niño entiende.
- **SpecializationEngine** calcula la producción de un turno. Especializado produce toda la capacidad; «hacer de todo» reparte el turno entre las k tareas que sabe y produce $\text{capacidad} / k$ de cada una (división entera: el tiempo partido se pierde). La lección de la especialización no se cuenta: se enseñan los dos resultados uno al lado del otro.
- **CooperationEngine** modela una cadena de etapas. El resultado es el **mínimo** de las capacidades por etapa, no la suma; y calcula también lo que conseguirían trabajando por separado, que es la comparación que da sentido a cooperar.
- **ScarcityEngine** valida repartos (nunca deja repartir más de lo que existe) y decisiones con presupuesto, y sabe enumerar **todas** las soluciones válidas: casi ningún reparto tiene una única respuesta correcta.
- **OpportunityCostEngine** calcula la renuncia exacta y la ventaja comparativa: no siempre debe producir algo quien mejor lo hace, sino quien menos pierde al dedicarse a ello.
- **ProductionChainEngine**, **LabEngine**, **ProgressEngine**, **DailyChallengeGenerator** y **ScenarioValidator** completan el conjunto.

6. Nunca una situación imposible

ScenarioValidator recorre cada desafío antes de que llegue a la pantalla y comprueba que los recursos existen, que las

cantidades son coherentes y —lo más importante— **que tiene al menos una solución**. Una prueba unitaria (`SeedContentTest`) ejecuta ese validador sobre los 42 desafíos: si alguien añade un escenario sin salida, la compilación de pruebas falla.

Además, `tools/verificar_escenarios.py` reimplementa las mismas reglas de forma independiente y comprueba una a una las soluciones previstas. Se ejecutó y pasó: los 42 desafíos tienen solución, todos los tratos propuestos como solución serían aceptados por el motor, y ningún escenario de escasez pide cubrir urgencias imposibles.

7. Progreso, recompensas y lo que se ha evitado

El progreso son **sellos del Valle**, y cada sello sale de una acción real guardada en la base de datos: un intercambio aceptado, un reparto de trabajo probado, un trabajo en equipo terminado, un experimento hecho. No existe ningún campo «puntuación» que se pueda inflar: todo se calcula contando filas.

Hay siete niveles con nombre (*Vecino nuevo* → *Guía del Valle*), 11 insignias, 24 objetos de colección en el Almacén del Valle y un reto diario generado localmente.

Deliberadamente **no hay**: rankings, comparación con otros niños, vidas, temporizadores obligatorios, compras, anuncios, castigos por no jugar, ni mensajes del tipo «inteligencia económica: 94 %». Ninguna insignia dice que el niño sea listo: todas describen algo que hizo.

Los errores no restan. El mensaje nunca es «incorrecto»: es «esta vez no hay coincidencia, ¿qué necesita el otro personaje?», con una pista y la posibilidad de volver a intentarlo. Los desafíos que se resistieron aparecen en **Practicar otra vez**, sin nota y sin diagnóstico.

8. Identidad visual

El Valle tiene su propio aspecto: verdes de prado, cremas de trigo, naranjas de teja y marrones de madera. Se evitó de forma consciente cualquier estética financiera adulta (nada de gráficos bursátiles, bancos, inversiones ni monedas).

Todas las ilustraciones están dibujadas con Compose Canvas dentro de la propia aplicación: 20 recursos, 9 personajes, 8 avatares, 13 edificios del Valle, 11 insignias, 13 iconos de conceptos y un paisaje de fondo con colinas, río y nubes. No se descarga ni una sola imagen; la app se ve exactamente igual sin conexión. Los iconos de lanzador son PNG generados localmente más un icono adaptativo vectorial.

El mapa principal no es una lista de botones: es el pueblo, con un camino de misiones que serpentea de arriba abajo, cada parada con su edificio ilustrado, su estado y su barra de avance.

9. Accesibilidad

- Ningún estado se expresa **solo** con color: cada estado lleva símbolo y palabra (`Bloqueado` ⛔, `Disponible` ▶, `Completado` ✓, `Dominado` ★), y lo mismo las urgencias (`!!! Lo necesita ya`).
- Las cantidades siempre se ven en número, junto al dibujo.
- Zonas táctiles grandes: los botones de cantidad miden 44 dp.
- El arrastre tiene alternativa: en la cadena de producción se puede tocar dos piezas para intercambiarlas o usar flechas arriba/abajo; en la mesa de trueques se toca el producto y se ajusta la cantidad con botones grandes.
- Opción de **texto más grande** en Ajustes, que escala toda la tipografía.
- Todos los dibujos llevan descripción para lectores de pantalla.

10. Privacidad infantil

La aplicación **no declara ningún permiso**, ni siquiera INTERNET. No hay backend, cuentas, analítica, anuncios, chat ni rankings en línea. No se pide nombre real, correo, teléfono, dirección, edad ni ubicación: solo un mote que el niño escribe si quiere (puede dejarlo vacío) y un avatar elegido entre ocho dibujos locales. Todo el progreso vive en `redeconomica.db`, en el dispositivo, y se puede borrar desde Ajustes.

11. Tecnología y arquitectura

Kotlin 2.0.21 · Jetpack Compose (BOM 2024.09.00) · Material 3 · Navigation Compose 2.8.0 · Room 2.6.1 con KSP · Coroutines y Flow/StateFlow · Gradle Kotlin DSL con catálogo de versiones · JDK 17 · minSdk 24 · targetSdk 34. Todas las versiones fijas, ninguna dinámica.

Separación en tres capas: `domain/` (modelos y motores, testeables sin interfaz), `data/` (Room, repositorios y contenido semilla) y `ui/` (Compose y ViewModels). La inyección de dependencias es un contenedor explícito escrito a mano (`AppContainer`): la app es pequeña, el grafo es plano y así se lee mejor y compila más rápido que con Hilt. El proyecto tiene 75 archivos Kotlin de aplicación (unas 12 100 líneas) y 12 de pruebas (unas 1 600 líneas).

12. Pruebas

136 pruebas unitarias JVM más una instrumentada de humo:

Archivo	Pruebas	Qué cubre
<code>InventoryTest</code>	12	Inventarios, imposibilidad de negativos, persistencia en texto
<code>TradeEngineTest</code>	14	Las siete reglas de aceptación, búsqueda de tratos, dobles intercambios
<code>SpecializationEngineTest</code>	12	Producción, «de todo» frente a especializar, planes válidos
<code>CooperationEngineTest</code>	10	Mínimo de la cadena, cuellos de botella, mejora real por cooperar
<code>ScarcityEngineTest</code>	14	Repartos válidos y múltiples, presupuestos y renunciaciones
<code>OpportunityCostEngineTest</code>	12	Renuncia exacta, frontera de producción, ventaja comparativa
<code>ProductionChainEngineTest</code>	7	Orden correcto, punto de rotura, barajado reproducible
<code>ProgressEngineTest</code>	13	Sellos, niveles, estados de misión, insignias y avance
<code>SeedContentTest</code>	13	Los 42 desafíos con el validador, integridad, dificultad creciente
<code>DailyAndLabTest</code>	12	Reto diario determinista y experimentos del laboratorio
<code>PersistenciaTest</code>	17	Room real con Robolectric: sembrado, progreso, contadores, reinicio

Casos límite cubiertos: inventario vacío, intercambio sin recursos, intercambio imposible, necesidad ya satisfecha, producción cero, cantidades negativas, doble intercambio, personaje sin capacidad, escenario sin solución, escenario con varias soluciones, alias vacío, sembrado repetido, reinicio de progreso y base de datos nueva.

13. Simplificaciones documentadas

Siguiendo la regla de no reducir en silencio ninguna funcionalidad, se declaran las tres decisiones que se apartan de la lectura más literal del encargo:

- Arrastrar y soltar → tocar para colocar.** Las mecánicas descritas como «arrastrar» se implementaron como *tocar para coger / tocar para soltar*, más botones de cantidad y flechas de orden. La interacción es equivalente (mover piezas entre zonas), es plenamente funcional, y además cumple el requisito de accesibilidad de ofrecer alternativa al arrastre. No se substituyó por una pregunta de opción múltiple.
- Las REGLAS de los escenarios viven en el módulo de contenido, no en columnas de la base de datos.** En Room está la ficha de cada escenario (id, misión, tipo, textos, dificultad, orden), y de ahí salen los listados y el progreso. Las reglas concretas —inventarios, capacidades, etapas, opciones de presupuesto— son estructuras con forma distinta según la mecánica y viven en `data/seed`, fuera de la interfaz, como datos validados por `ScenarioValidator` y recorridos por las pruebas. Guardarlas en columnas habría exigido un serializador propio sin ganar ninguna consulta útil.
- Sonido y vibración: interruptores reales, efectos no incluidos.** Los ajustes de sonido y vibración existen, se guardan y se respetan, pero la versión 1.0.0 no incluye archivos de audio: la app es completamente silenciosa. Se prefirió no añadir sonidos genéricos a incluirlos sin haber podido probarlos en un dispositivo.

14. Estado de la compilación

COMPILACIÓN NO VERIFICADA. El entorno de generación no dispone de Android SDK ni de acceso a los repositorios de artefactos (`dl.google.com`, `services.gradle.org`, Maven Central), de modo que no se pudo ejecutar `./gradlew clean, testDebugUnitTest, lintDebug` ni `assembleDebug`, y **no se entrega APK**. Sí se verificaron: la coherencia y solubilidad de los 42 escenarios (script independiente), la carga real de `schema.sql` y `sample_data.sql` en SQLite, y una revisión estática de referencias e importaciones. El detalle honesto de qué se comprobó y qué no está en `docs/BUILD_REPORT.md`.

El proyecto incluye `.github/workflows/build.yml` para que, al subirlo a GitHub, Actions ejecute las pruebas, el lint y `assembleDebug` y publique el APK como artefacto.